



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии» (ИУ7)

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2

Название: Записи с вариантами. Обработка таблиц

Дисциплина: Типы и структуры данных

Студент

ИУ7-31Б

(Группа)

Постнов С.А

(Фамилия И. О.)

Преподаватель

Барышникова М. Ю.

Оглавление

Условие задачи	3
Описание ТЗ.....	3
1. Описание входных данных	3
2. Описание результатов работы программы	4
3. Способ обращения к программе	5
4. Задача программы	5
5. Описание возможных аварийных ситуаций и ошибок пользователя....	6
Описание внутренних структур данных	6
Тестовые случаи	7
Сравнение эффективности алгоритмов и сортировок.....	8
Выводы	9
Ответы на контрольные вопросы	9

Условие задачи

Создать таблицу, содержащую не менее 40-ка записей (тип – запись с вариантами (объединениями)). Упорядочить данные в ней по возрастанию ключей, двумя алгоритмами сортировки, где ключ – любое невариантное поле (по выбору программиста), используя: 1) саму таблицу 2) массив ключей. Возможность добавления и удаления записей в ручном режиме обязательна. Осуществить поиск информации по варианту. Ввести список машин, имеющихся в автомагазине, содержащий: марку автомобиля, страну-производитель, цену, цвет и состояние:

- новый – гарантия (в годах);
- нет – год выпуска, пробег, количество ремонтов, количество собственников.

Вывести цены не новых машин указанной марки с одним предыдущим собственником, отсутствием ремонта в указанном диапазоне цен.

Описание ТЗ

1. Описание входных данных

Таблица данных представлена информацией об автомобилях, в каждой записи которой располагается:

1. Марка автомобиля
2. Страна-производитель
3. Цена
4. Цвет
5. Состояние
6. Гарантия (если состояние – новое)
7. Год выпуска (если состояние – б/у)
8. Пробег (если состояние – б/у)
9. Количество ремонтов (если состояние – б/у)
10. Количество собственников (если состояние – б/у)

Ограничения:

1. Марка машины содержит не более **11 символов**
2. Страна-производитель содержит не более **15 символов**
3. Цена автомобиля содержит не более **8 цифр**
4. Цвет автомобиля содержит не более **11 символов**
5. Состояние автомобиля определяется следующим образом:
 - Новое, если **0**
 - Б/у, если **1**
6. Гарантия содержит не более **2 цифр**
7. Год выпуска содержит не более **4 цифр**
8. Пробег содержит не более **6 цифр**
9. Количество ремонтов содержит не более **2 цифр**
10. Количество собственников содержит не более **2 цифр**
11. Файл на входе может быть пустым

2. Описание результатов работы программы

В результате работы программы будет создана база данных (таблица) с возможностью работать с различными пунктами меню:

1. Поиск информации о новых автомобилях указанной марки с одним предыдущим собственником, отсутствием ремонта в указанном диапазоне цен
2. Упорядочить таблицу автомобилей по возрастанию цены на автомобиль и вывести
3. Упорядочить таблицу ключей по возрастанию цены на автомобиль и вывести
4. Вывести исходную таблицу
5. Вывести таблицу ключей
6. Вывести исходную таблицу в упорядоченном виде, используя таблицу ключей

7. Добавить запись в конец таблицы
8. Удалить записи по значению цены
9. Просмотр отсортированной таблицы ключей при неотсортированной исходной таблице
10. Произвести замеры эффективности обычной и ускоренной сортировок при обработке данных в исходной таблице и таблице ключей
11. Вернуть таблицу ключей в исходное состояние
0. Выход

3. Способ обращения к программе

Обращение к программе происходит из командной строки путем запуска исполняемого файла «*app.exe*».

4. Задача программы

Задача данной программы несла в себе несколько составных частей:

1. Работа с базой данных посредством добавления/удаления записей
2. Форматированный вывод таблиц
3. Сортировка данных при помощи вспомогательной таблицы ключей и без
4. Количественное сравнение эффективности разных алгоритмов сортировки
5. Количественное сравнение памяти затрачиваемой на работу оперативной памяти
6. Определение разницы между ускоренной и обычной сортировками

5. Описание возможных аварийных ситуаций и ошибок пользователя

- Пользователь ввел некорректное имя файла:
 - Код возврата: 1
 - Сообщение: «Не удалось открыть файл. Попробуйте снова.»
- Файл содержит слишком много записей:
 - Код возврата: 3
 - Сообщение: «Произошло переполнение таблицы. Освободите место и попробуйте снова.»
- Некорректное поле в файле:
 - Код возврата: 4
 - Сообщение: «Не удалось прочитать файл с данными. Попробуйте снова.»
- Пользователь ввел некорректный код для меню:
 - Код возврата: 5
 - Сообщение: «Некорректный код. Попробуйте снова.»

Описание внутренних структур данных

Тип данных, описывающий структуру для хранения информации об автомобиле:

```
typedef struct info
{
    char model[MODEL_LEN + 2]; // Марка
    char country[COUNTRY_LEN + 2]; // Страна
    int price; // Цена
    char color[COLOR_LEN + 2]; // Цвет
    int condition; // Состояние
    union
    {
        int guarantee; // Гарантия
        struct old_info
        {
            int year; // Год выпуска
            int mileage; // Пробег
            int number_of_repairs; // Кол-во ремонтов
            int number_of_owners; // Кол-во собственников
        } old_car;
    };
};
```

```
    } add_info;  
  
} car_info;
```

Тип данных, описывающий структуру для хранения ключей к исходной таблице с выбранным полем:

```
typedef struct indexes  
{  
    int index; // Индекс  
    int price; // Цена  
} help_table;
```

Тип данных, описывающий структуру для хранения результатов измерения:

```
typedef struct measure  
{  
    long long unsigned time; // Время работы  
    int mem; // Выделенная память  
} measurement_table;
```

Тестовые случаи

Позитивные тесты:

1. Сортировка одной записи
2. Сортировка одинаковых записей
3. Сортировка отсортированных записей
4. Поиск нескольких записей
5. Поиск одной записи
6. Поиск записи без нахождения
7. Добавление записи в пустой файл/массив
8. Удаление единственной записи

Негативные тесты:

1. Некорректная запись
2. Некорректное имя файла
3. Файл содержит много записей
4. Неверный код выбора пункта меню

Сравнение эффективности алгоритмов и сортировок

Таблица замеров при 5000 записей в файле (в микросекундах, по результатам 10 запусков каждого алгоритма):

	SLOW SORT DATA	QUICK SORT DATA	SLOW SORT KEYS	QUICK SORT KEYS
TIME, mcs	1037916	535	158619	418
MEMORY, b	360000	360000	400000	400000

Исходя из результатов в представленной выше таблице, можно сделать вывод, что ускоренная сортировка быстрее обычной примерно в 1180 раз. Причем, заметим, что для меньшей размерности, процент падает, а для большой – наоборот повышается:

Таблица замеров при 3000 записей в файле (в микросекундах, по результатам 10 запусков каждого алгоритма):

	SLOW SORT DATA	QUICK SORT DATA	SLOW SORT KEYS	QUICK SORT KEYS
TIME, mcs	364079	295	56235	232
MEMORY, b	216000	216000	240000	240000

В этом случае ускоренная сортировка быстрее обычной примерно в 725 раз.

Таблица замеров при 7000 записей в файле (в микросекундах, по результатам 10 запусков каждого алгоритма):

	SLOW SORT DATA	QUICK SORT DATA	SLOW SORT KEYS	QUICK SORT KEYS
TIME, mcs	2002370	785	314972	722
MEMORY, b	504000	504000	560000	560000

В этом случае ускоренная сортировка быстрее обычной примерно в 1500 раз.

Также заметим, что сортировка таблицы ключей позволяет примерно на 16% получить выигрыш во времени работы алгоритма для простой сортировки и примерно 25% для ускоренной. Однако памяти требуется немного больше для хранения таблицы ключей (на 10%).

Выводы

В ходе работы были изучены способы и алгоритмы работы с базами данных, а также проведены замерные эксперименты, по результатам которых можно сказать, что использование дополнительной памяти (для таблицы ключей) в размере 10% от основной уменьшает время работы сортировки примерно на 25 % - для ускоренной, 16% - для обычной.

Ответы на контрольные вопросы

- 1) Объединенные поля (вариантные) занимают одну область памяти, поэтому память выделяется по размеру наибольшего из полей.
- 2) Некорректное представление и вывод данных.
- 3) Программист.
- 4) При больших размерах таблиц поиск данных и их сортировка может потребовать больших затрат времени. В этом случае можно время обработки за счет создания дополнительного массива – таблицы ключей, содержащей индекс элемента в исходной таблице и выбранный ключ.
- 5) Если мы сортируем таблицу ключей, то экономится время, поскольку перестановка записей в исходной таблице, которая иногда может содержать достаточно большое число полей, отсутствует. Этот выигрыш во времени особенно заметен при большой размерности таблиц и при правильно подобранных ключах. Но для размещения таблицы ключей требуется дополнительная память. А также, если в качестве ключа используется символьное поле записи, то это влечет за собой необходимость посимвольной обработки данного поля в

цикле, и, следовательно, приводит к увеличению времени выполнения любых операций.

- 6) Сортировки, обладающие свойством устойчивости, так как перестановка одинаковых элементов не имеет смысла, но занимает достаточно много времени и средств.